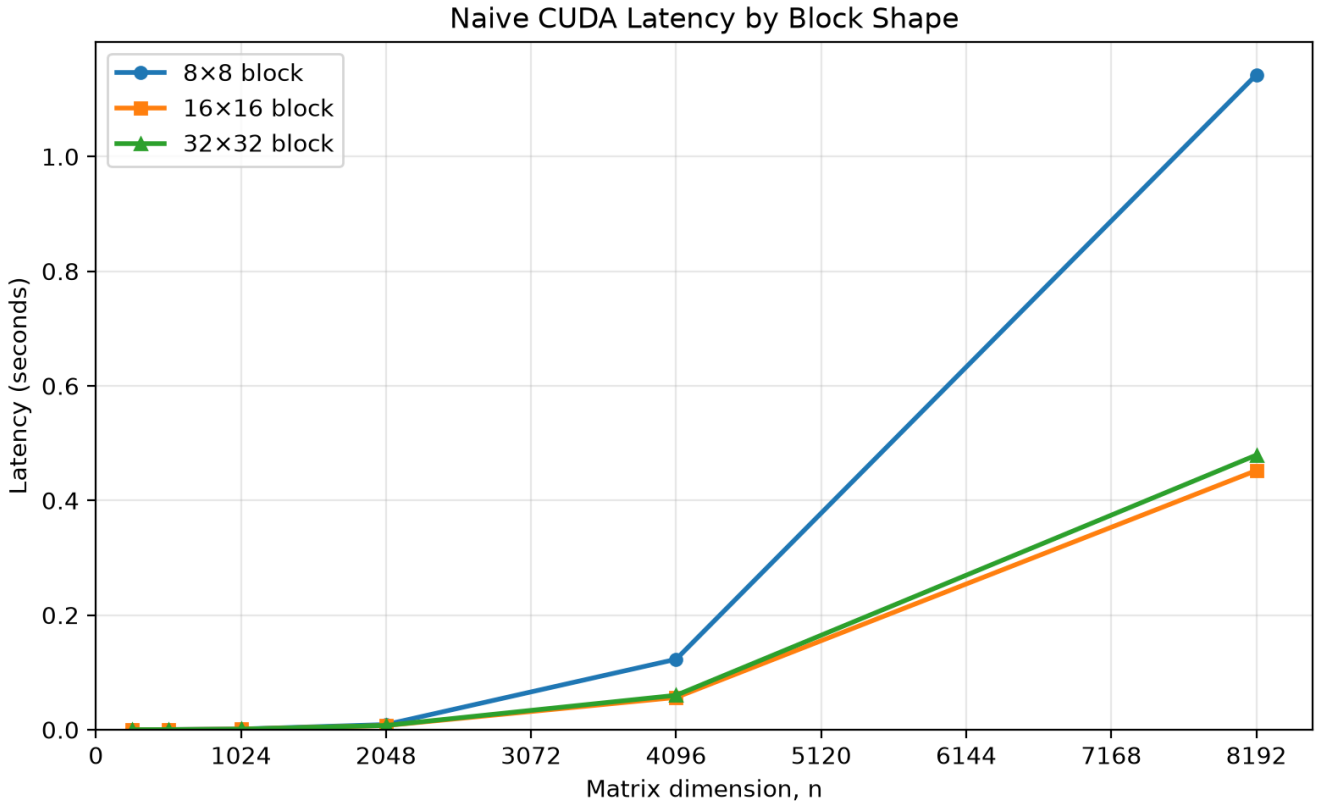
# Section Overview

- Memory Boundedness, Basic Tiling, and Analysis

# Let's do some experimenting

**This kernel assigns one thread per output element, and computes them independently. So changing block size shouldn't matter right?**



# Blocks and SMs

## Interpretation

This happens because entire **blocks** are scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

# Blocks and SMs

## Interpretation

This happens because entire **blocks** are scheduled to SM's. The larger block sizes of 16x16 and 32x32 lead to more **L1 cache hits**.

| **Block shape** | **L1 load hit rate** |
|---|---|
| $8 \times 8$ | 83.51% |
| $16 \times 16$ | 87.13% |
| $32 \times 32$ | 94.98% |

RTX 3090, $M = N = K = 4096$

# Shared Memory (SMEM) Caching

## Programmable L1 Cache (SMEM)

Remember how our launch config required us to group threads as blocks? That is because **blocks** are scheduled to SM's as a unit, so **threads in the same block have access to the SM's L1 cache/SMEM.**

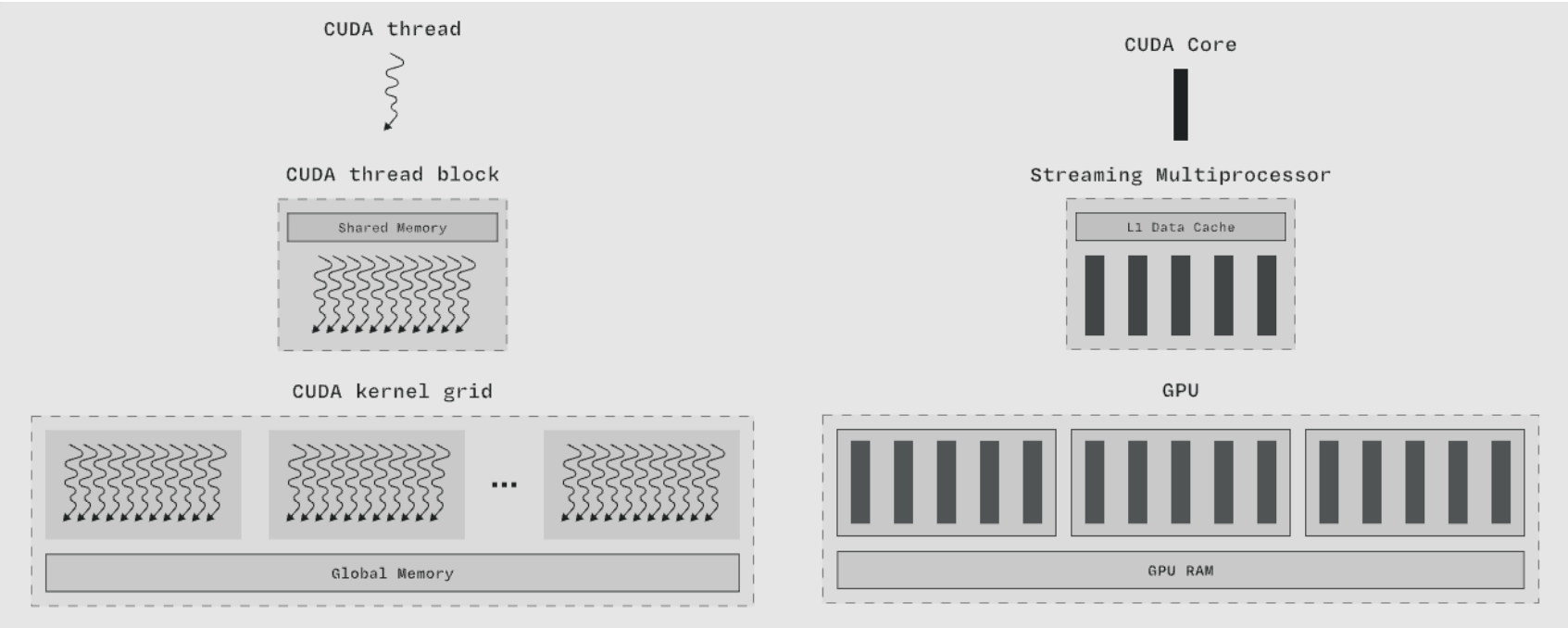



Image source: Modal, "CUDA Programming Model," *GPU Glossary*.

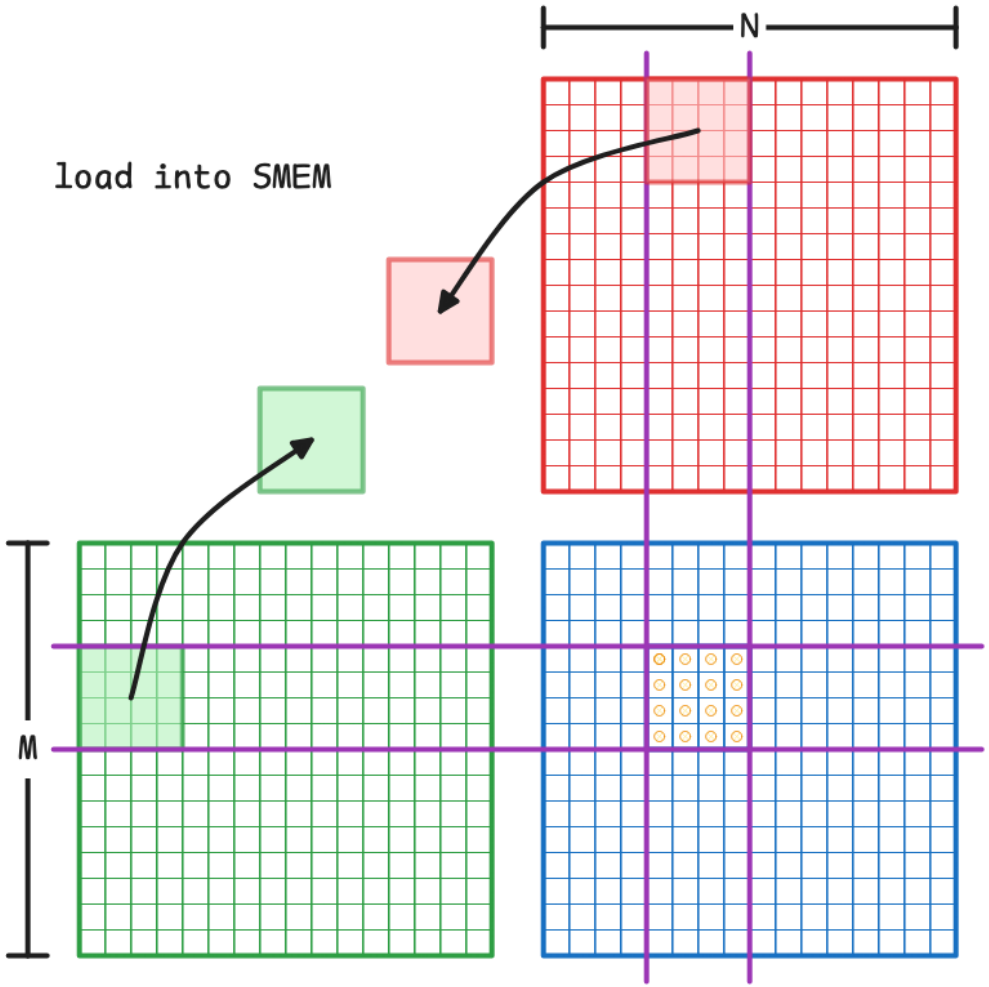
# Tiling

Threads can save time by loading data from Device/Global memory into Shared Memory within the SM.

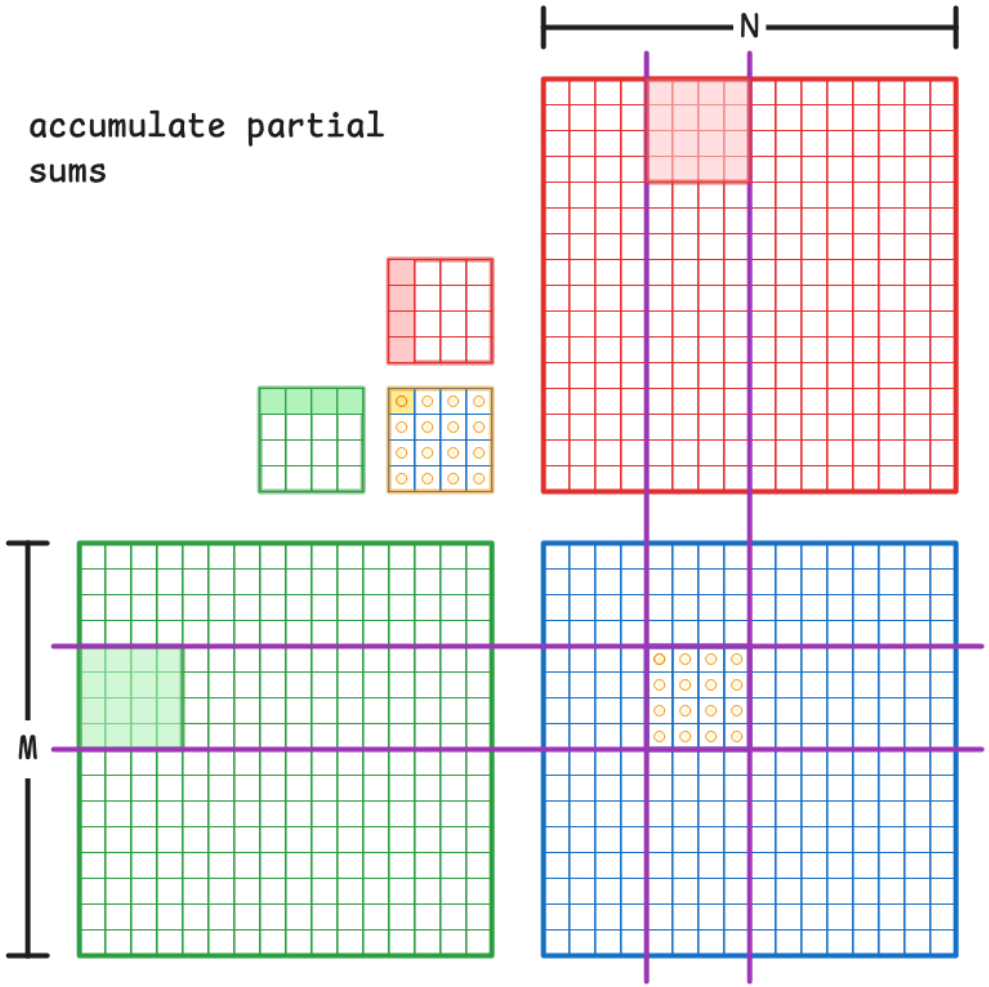
## Main Idea

Before doing any computation, load a chunk of the necessary data from DRAM to SMEM as a block. Then, memory accesses for the computation now only go to SMEM.

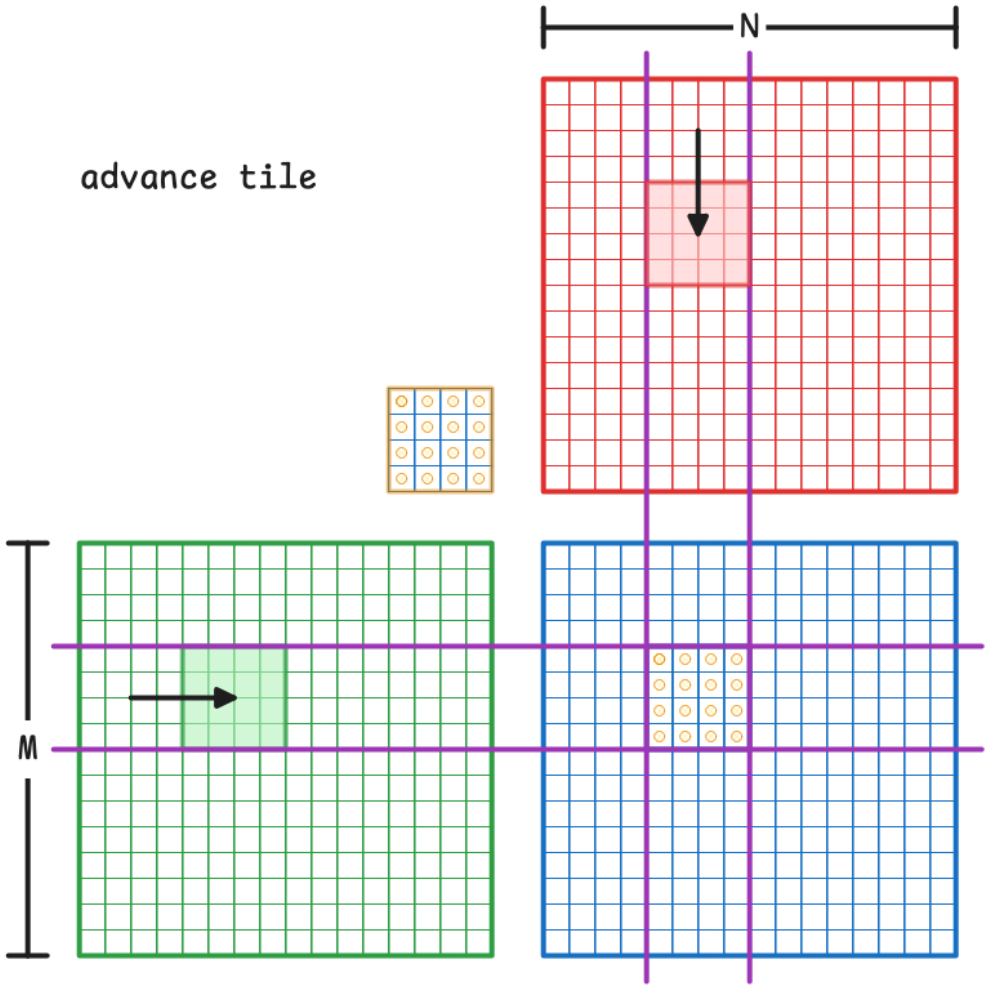
# Tiling steps



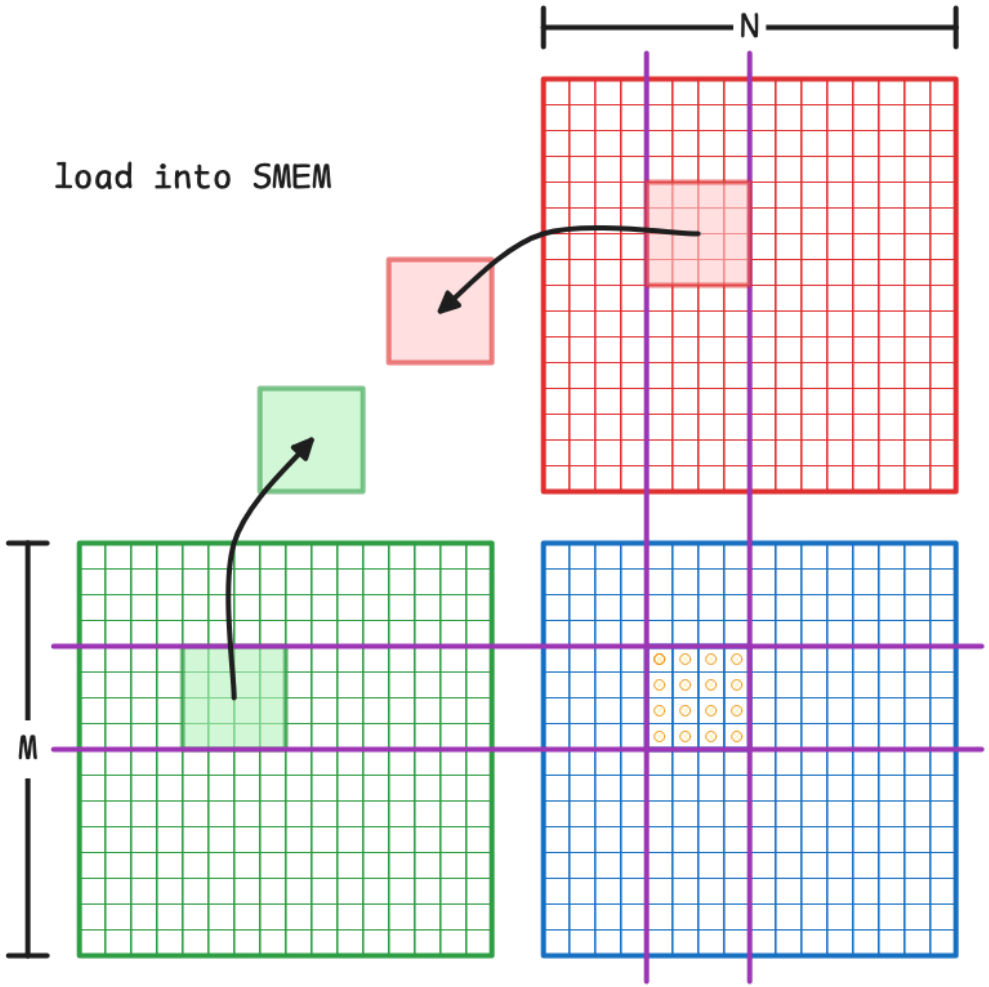
# Tiling steps



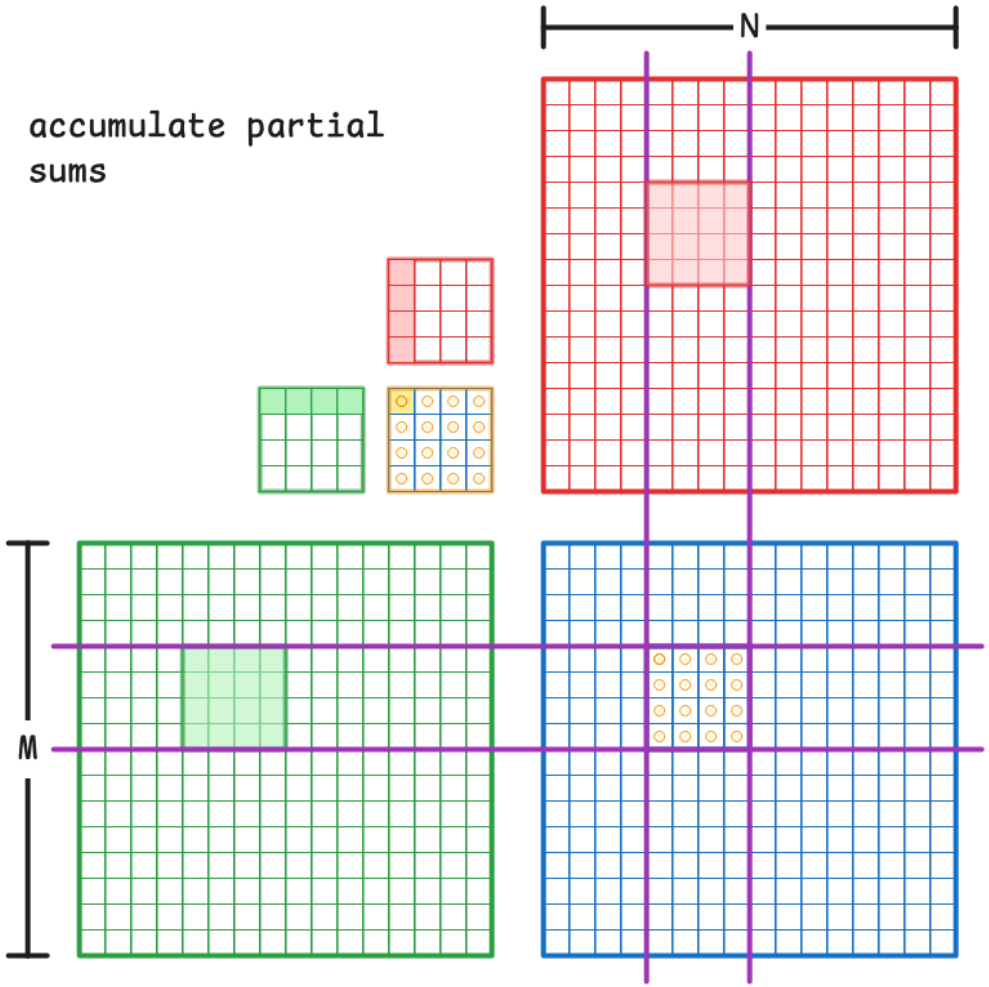
# Tiling steps



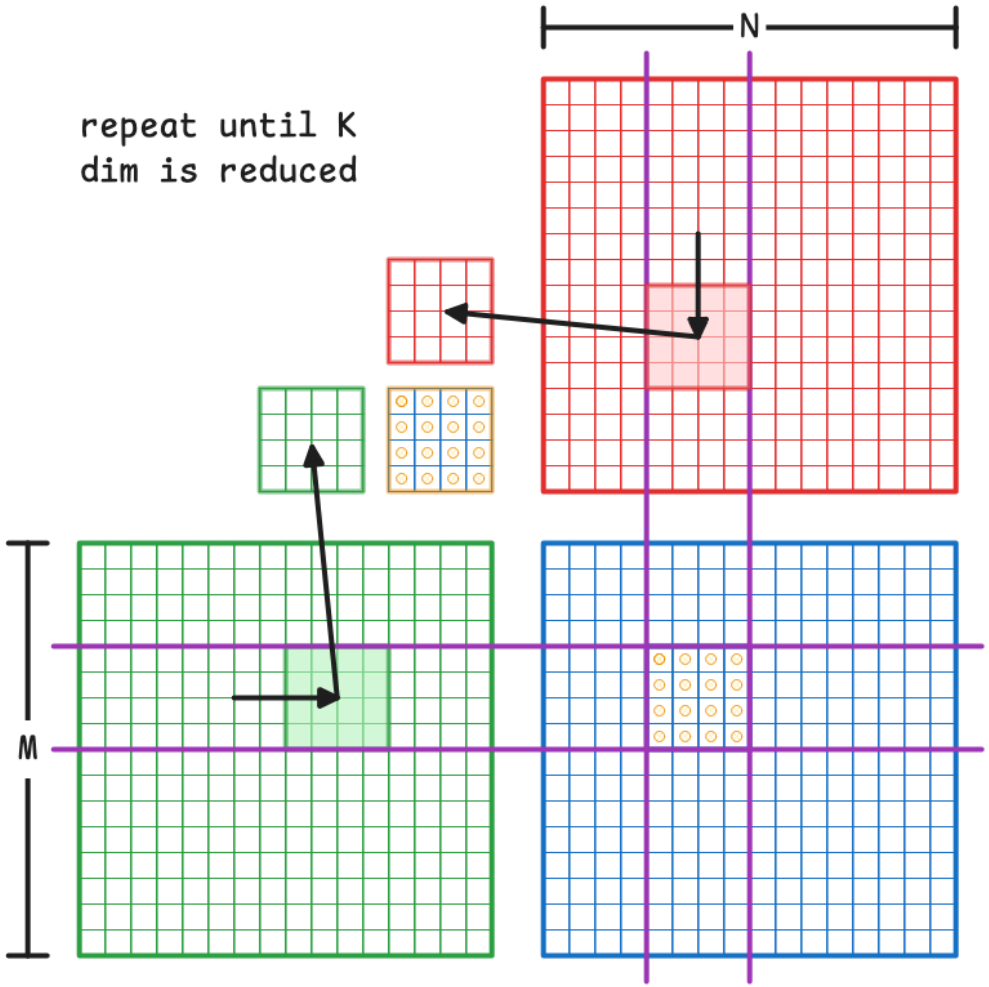
# Tiling steps



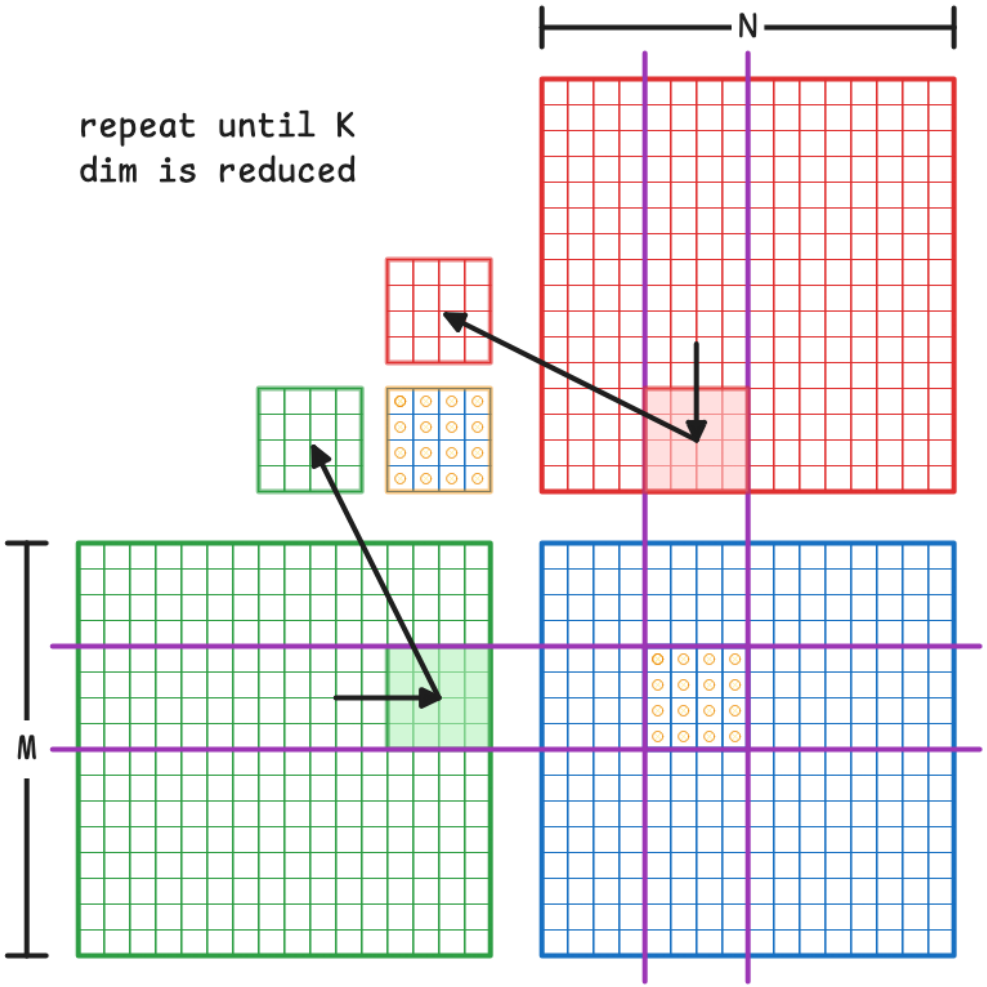
# Tiling steps



# Tiling steps



# Tiling steps



# Basic tiled SGEMM

```
__shared__ float As[BM * BK];
__shared__ float Bs[BK * BN];
float tmp = 0.0;

for (int bkIdx = 0; bkIdx < K; bkIdx += BK) {
  As[threadRow * BK + threadCol] =
      A[threadRow * K + threadCol];
  Bs[threadRow * BN + threadCol] =
      B[threadRow * N + threadCol];

  __syncthreads();
  A += BK;
  B += BK * N;

  for (int dotIdx = 0; dotIdx < BK; ++dotIdx) {
    tmp += As[threadRow * BK + dotIdx] *
           Bs[dotIdx * BN + threadCol];
  }

  __syncthreads();
}
```

**Cooperate**
Each thread loads one value from $A$ and one from $B$.

**Synchronize**
Wait until both shared tiles are complete.

**Reuse**
One thread accumulates from the cached row and column.

**Advance**
Protect the cache, move along $K$, and repeat.

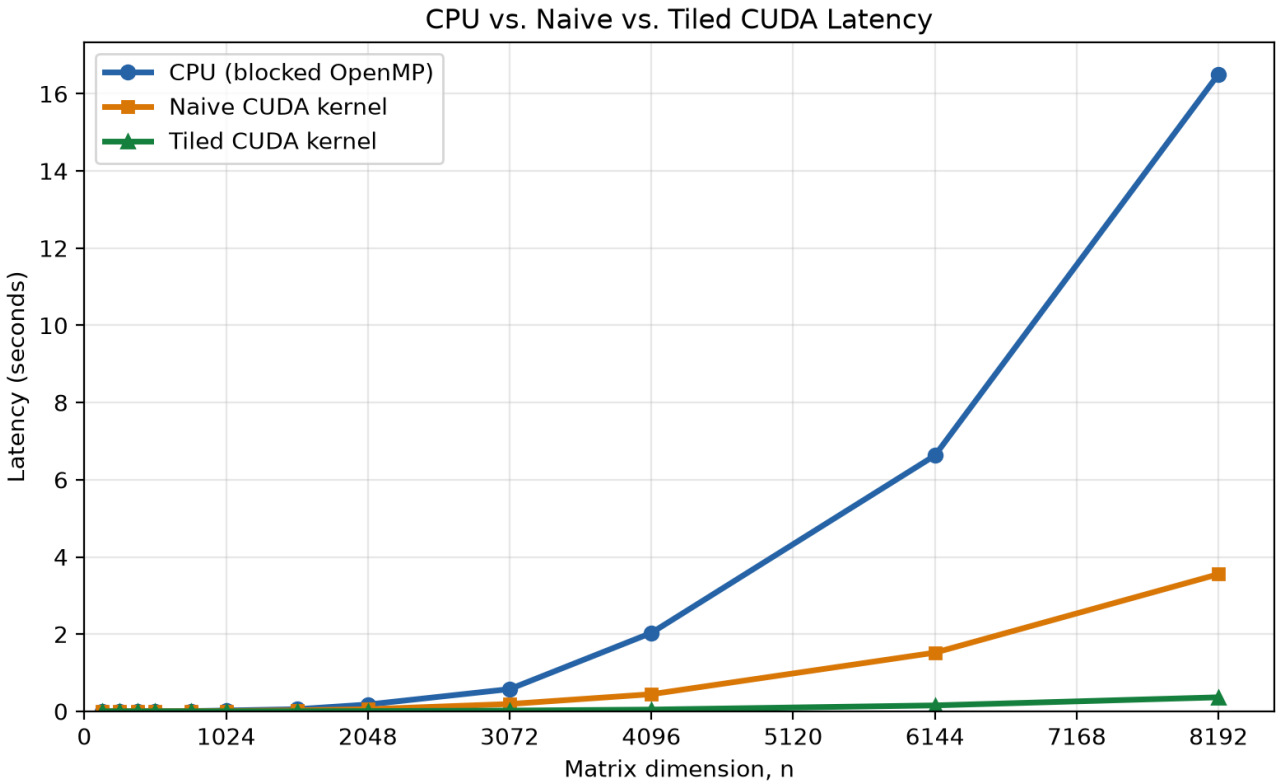
Adapted from Boehm (2022), kernel 3.

# Basic tiled SGEMM visualization

- Watch each block load small regions of $A$ and $B$ together.
- The loaded tiles are reused to update an entire tile of $C$.
- Synchronization separates loading, computing, and advancing to the next tiles.

Play tiled-kernel visualization

# Tiling cuts latency by another order of magnitude



# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM.

# Can we still do better?

## Throughput analysis

We report a final latency of 0.36s or 360ms for $n, m, k = 8192$. This is around $10\times$ the theoretical minimum of 31ms. This slowdown suggests we are still **memory bound**.

This tiling setup with $32 \times 32$ blocks and $n, m, k = 8192$, we need a **grid** of $\frac{m}{32} \times \frac{n}{32} = 256 \times 256 = 65,536$ blocks.
Each block loads $\frac{k}{32} = 256$ tiles from both $A$ and $B$. Each tile is $32 \times 32$ elements.
Thus, we have

$$\underbrace{65,536}_{\text{blocks}} \times \underbrace{256}_{\text{tiles/block}} \times \underbrace{2}_{A \text{ and } B} \times \underbrace{32 \times 32}_{\text{elements/tile}} \times \underbrace{4 \text{ bytes}}_{\text{bytes/element}} \approx 137.7 \text{ GB}.$$

At 935.8 GB/s, we need 0.147s or 147ms to move all data from DRAM to SMEM. **40% of our latency is moving data.**